

Web アプリ チーム開発

はじめての一步

GDG on Campus University of Osaka
2 時間で「チームに参加できる」状態へ

この研修の目的



ゴール

経験者に助けてもらいながら、
AI も使いつつ、

**最低限チーム開発に参加できる
状態になる。**

2026 年の優先順位

HTML / CSS 文法より先に、これらを身につけよう。

- AI への質問能力
- エラー共有能力
- 状況説明能力

コードを書ける < 助けを受けて安全に開発できる

全体構成（120 分）

#	内容	時間
1	Web アプリ全体像	20 分
2	開発フロー体験	35 分
3	Git 事故対策	15 分
4	開発者ツール & コンソール	20 分
5	AI への聞き方	20 分
6	チームルール	10 分

01. Web アプリ全体像

20 分

このセクションのゴール

詳細実装ではなく、

自分がどこを触っているか分かる

状態を目指す。

Web アプリの構成

データは上から下へ流れる:

- **Browser** (ブラウザ)
- **Frontend** (画面)
- **Backend** (処理)
- **DB** (保存)

Frontend の役割

ユーザーが**直接**触れる部分。

- 画面の表示
- ボタンのクリック
- フォームの入力

例: React, HTML / CSS, JavaScript

Backend の役割

Frontend からの依頼を処理する。

- データ処理
- DB / Storage との接続
- ログイン / API 提供

例: Node.js, Python, Go

DB (データベース)

データを保存する場所。

- ユーザー情報
- 投稿内容
- 設定

消えると困るものはDB へ。

GitHub の役割

チーム開発の中心地。

- コードの共有
- PR レビュー
- 変更履歴の管理

Docker（概念だけ）

みんなの開発環境を揃える箱。

- 「うちでは動くのに…」を防ぐ
- 今は使い方を覚えなくて OK
- 「箱で揃える仕組みがある」だけ知る

ここで大事なこと

詳細実装ではなく、

データがどう流れるか

を理解しよう。

「自分が今どこを触っているか」 がいえれば合格！

02. 開発フロー体験

35 分（メイン）

このセクションのゴール

全員で実際に手を動かす。

- clone → 起動 → 修正 → commit → push → PR
- 初心者最大の壁「変更 → PR」を経験する
- 経験者は隣でサポート

Step 1. clone する

リポジトリを自分の PC に持ってくる。

```
git clone https://github.com/xxx/yyy.git  
cd yyy
```

URL はチームから配布されます。

Step 2. 起動する

依存パッケージを入れて開発サーバーを立ち上げる。

```
npm install  
npm run dev
```

ブラウザで <http://localhost:3000> を開く。

⚠️ スクリーンショット

`npm run dev` 成功時のターミナル出力

(プレースホルダー)

Step 3. 文言を修正

例: ページのタイトルを変えてみよう。

```
- <h1>Hello</h1>  
+ <h1>Hello World</h1>
```

ブラウザをリロードして変わっているか確認！

Step 4. commit する

変更内容を Git に記録する。

```
git add .  
git commit -m "fix: change title text"
```

コミットメッセージは「**何をしたか**」を短く。

Step 5. push する

自分の変更を GitHub に送る。

```
git push
```

⚠️ スクリーンショット

GitHub の Pull Request 作成画面

(プレースホルダー)

Step 6. PR を作成

GitHub 上で **Pull Request** を作る。

- ブラウザで GitHub を開く
- 「Compare & pull request」 ボタン
- タイトルと説明を書いて作成

このセクションのゴール（再）

変更 → PR を一通り経験できた！

- 最初は緊張するが、誰もが通る道
- 詰まったら即メンターに声かけ
- 「動いた」の感覚を覚えておこう

03. Git 事故对策

15 分

このセクションのゴール

完璧に使いこなすより、まず

壊し方を知る。

事故を踏まないための最低限を学ぶ。

ルール 1. main で作業しない

必ずブランチを切ってから作業。

```
git switch -c feature/my-task
```

feature/<やること> の形が分かりやすい。

ルール 2. 作業前に pull

最新を取ってきてから作業を始める。

```
git pull
```

古い状態で作業すると *conflict* の原因。

ルール 3. 困ったら git status

今の状態を確認するクセを付ける。

```
git status
```

- どのファイルを変えたか
- どこまで commit したか
- どのブランチにいるか

ルール 4. conflict は助けを呼ぶ

merge conflict は初心者の鬼門。

- 無理に直そうとしない
- そのまま経験者に画面を見せる
- 学ぶのは慣れてから

禁止事項

これだけは絶対やらない。

```
git push --force          # ✗ 履歴を吹き飛ばす  
git push --force-with-lease # ✗ こちらもまず使わない
```

やるなら**必ず**経験者に確認。

04. 開発者ツール & コンソール

20 分

なぜ大事？

AI 活用の前提知識。

- エラーが読めれば AI に渡せる
- 渡せれば直せる
- 直せれば自走できる

4-1. ブラウザ開発者ツール



10 分

開き方

どのブラウザでも共通。

- **F12** キー
- 右クリック → 「検証」
- macOS: **Cmd + Option + I**

⚠️ スクリーンショット

Chrome DevTools 全体像（タブが見える状態）
（プレースホルダー）

Console タブ

JavaScript のエラーと出力を見る。

- 赤文字を探す
- まずは 1 行目を読む
- 例: `Uncaught TypeError: ...`

⚠️ スクリーンショット

Console タブにエラーが表示された画面
(プレースホルダー)

Network タブ

API 通信の様子を見る。

- リクエスト一覧
- **赤色 / 404 / 500** を探す
- 「サーバーまで届いてる？」を確認

⚠️ スクリーンショット

Network タブで失敗リクエストが赤く出ている画面
(プレースホルダー)

Elements タブ

HTML の構造を見る。

- 今表示されている HTML
- CSS が当たっているか
- クラス名は何か

最重要ポイント

エラー文を読む

文化を作ろう。

「赤いの出てます」ではなく、「〇〇というエラーが出ました」。

4-2. ターミナル / コンソール

10 分

ターミナルは怖くない

初心者はターミナル恐怖症になりやすい。

まず覚えてほしいことは1つ:

壊れても大抵戻せる

最低限コマンド: pwd

今いる場所を確認する。

```
pwd
```

「自分がどこにいるか」を言えるのが第一歩。

最低限コマンド: ls / dir

その場のファイル一覧を見る。

```
ls          # macOS / Linux  
dir         # Windows PowerShell
```

最低限コマンド: cd

別のフォルダに移動する。

```
cd folder-name  
cd ..          # 一つ上に戻る
```


最低限コマンド: Ctrl + C

動いているコマンドを止める。

```
Ctrl + C
```

- `npm run dev` を止めるとき
- 反応がなくなったとき
- これが効くと知るだけで安心感が違う

⚠️ スクリーンショット

ターミナルで `pwd / ls / cd` を実行した出力例
(プレースホルダー)

伝えたいこと

ターミナルは「黒い画面」ではなく、
コンピュータとの対話窓口。

壊れても大抵戻せる。怖がらず触ろう。

05. AI への聞き方

20 分

なぜ必須？

2026 年において、AI を使いこなせるかどうかは
生産性に直結するスキル。

- 教科書を読む前に AI に聞いてもいい
- ただし聞き方で結果が大きく変わる

5-1. AI に渡す情報



悪い例

動きません

- AI もエスパーではない
- 状況不明 → 当てずっぽうの回答
- 結局解決しない

良い例

React で npm run dev したら
Module not found が出ました

【やりたいこと】
ログイン画面を表示したい

【エラー全文】
xxx

【試したこと】
npm install をやり直した

良い質問の 3 要素

- やりたいこと（目的）
- 起きていること（エラー全文）
- 試したこと（やった対応）

この3つを毎回書くだけで劇的に変わる。

5-2. スクショを貼る重要性



初心者のやりがちなパターン

エラーを要約してしまう。

なんか赤いエラー出てます

→ AI には何も伝わらない。

AI 時代の正解

スクショをそのまま貼る。

- レイアウト崩れ
- ブラウザの表示
- エラー画面
- DevTools
- ターミナル

なぜ強い？

AI は画像認識が強い。

- ChatGPT
- Claude
- Gemini

文字起こしより、画像をそのまま投げる方が早くて正確。

5-3. スクショの撮り方



Windows

全画面

PrintScreen

範囲選択

Windows + Shift + S

63

⚠ スクリーンショット

Windows の「切り取り & スケッチ」ツールが起動している画面
(プレースホルダー)

macOS

範囲選択

Cmd + Shift + 4

全画面

Cmd + Shift + 3

65

⚠️ スクリーンショット

macOS の範囲選択スクショ（カーソルが十字になっている状態）
（プレースホルダー）

5-4. AI を盲信しない



AI は平気で嘘をつく

それっぽいコードを返すが、

- 存在しない関数を使う
- 古い API を提案する
- 副作用を説明しない

ことがある。

特に危険な操作

これらは AI 任せにしない:

- ライブラリの追加 / 削除
- DB の操作
- 設定ファイルの大きな変更
- `--force` 系コマンド

基本ルール

大きい変更の前は経験者に聞く。

- 「ライブラリ入れていい？」
- 「この config 触っていい？」
- 「DB のテーブル変えていい？」

1 分の確認で半日のロスが防げる。

06. チームルール

10 分

なぜ必要？

学生プロジェクトはここで崩壊しやすい。

- 仲が悪いから崩れるのではない
- ルールがないから崩れる
- 最低限の約束を共有する

ルール 1. 詰まったら 30 分で相談

- 1人で抱え込まない
- 30分やって進まなければ声をかける
- 「相談する」は弱さではなくスキル

ルール 2. エラー文を貼る

「動きません」は禁止ワード。

- エラー全文を貼る
- スクショを貼る
- 状況を一言添える

ルール 3. 勝手にライブラリを追加しない

- `npm install xxx` の前に相談
- 依存が増えると全員に影響
- セキュリティ的にも要確認

ルール 4. 勝手に main へ push しない

- 必ずブランチで作業
- PR を出してレビューを通す
- 直 push は事故のもと

ルール 5. 壊したら隠さない (最重要)

- 「やっちゃいました」を即報告
- 早ければ早いほどリカバリは簡単
- 隠すと被害が広がる

隠さないチームは強い。

おまけ

この2時間で「教えないこと」

あえて省いた内容

今日は触れません:

- React の詳細
- TypeScript の型を厳密に理解
- CSS 設計
- DB 理論
- Docker の操作
- アルゴリズム / 設計原則

なぜ省いた？

最初に全部やると全部消化不良になる。

- 必要になったときに学ぶ方が定着する
- まず「参加できる」を優先
- その先は実戦で身につく

まとめ



この研修の本質

重要なのは、

コードを書けることではなく、

助けを受けながら安全に開発参加できること。

今日から意識すること

- 自分がどこを触っているか言える
- エラーをそのまま共有する
- AI に状況を渡せる
- 困ったらすぐ相談する
- 壊したら隠さない

質問タイム



ようこそ、チームへ！